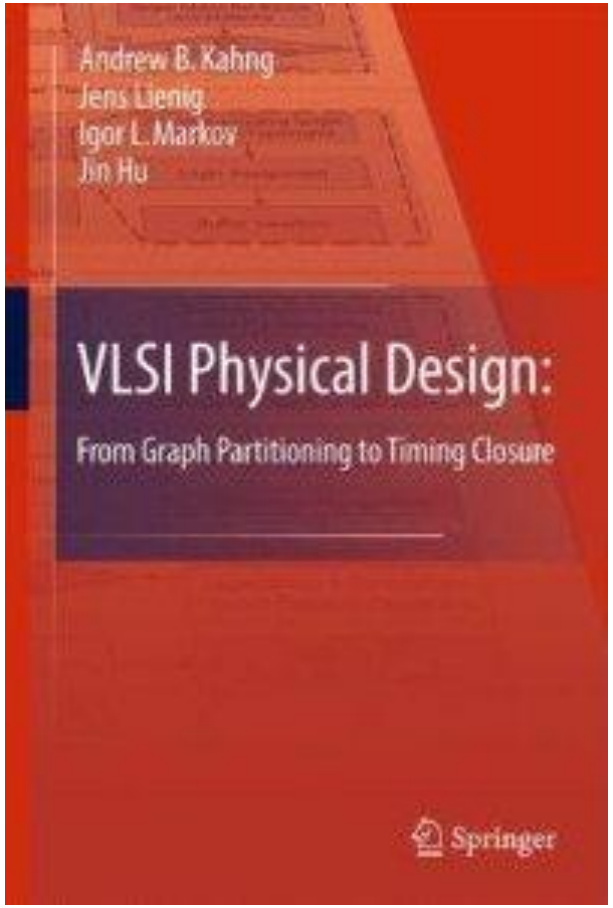
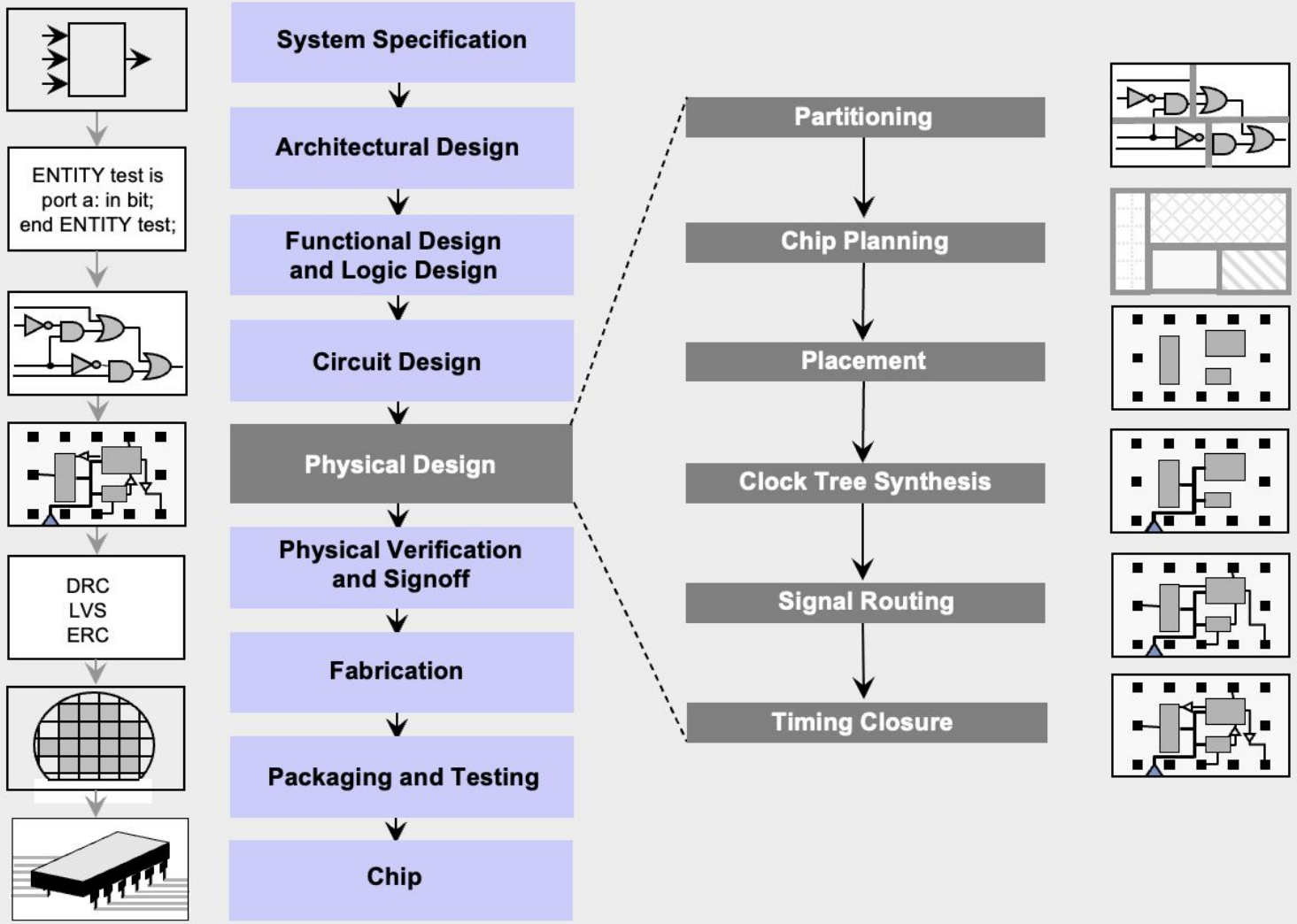


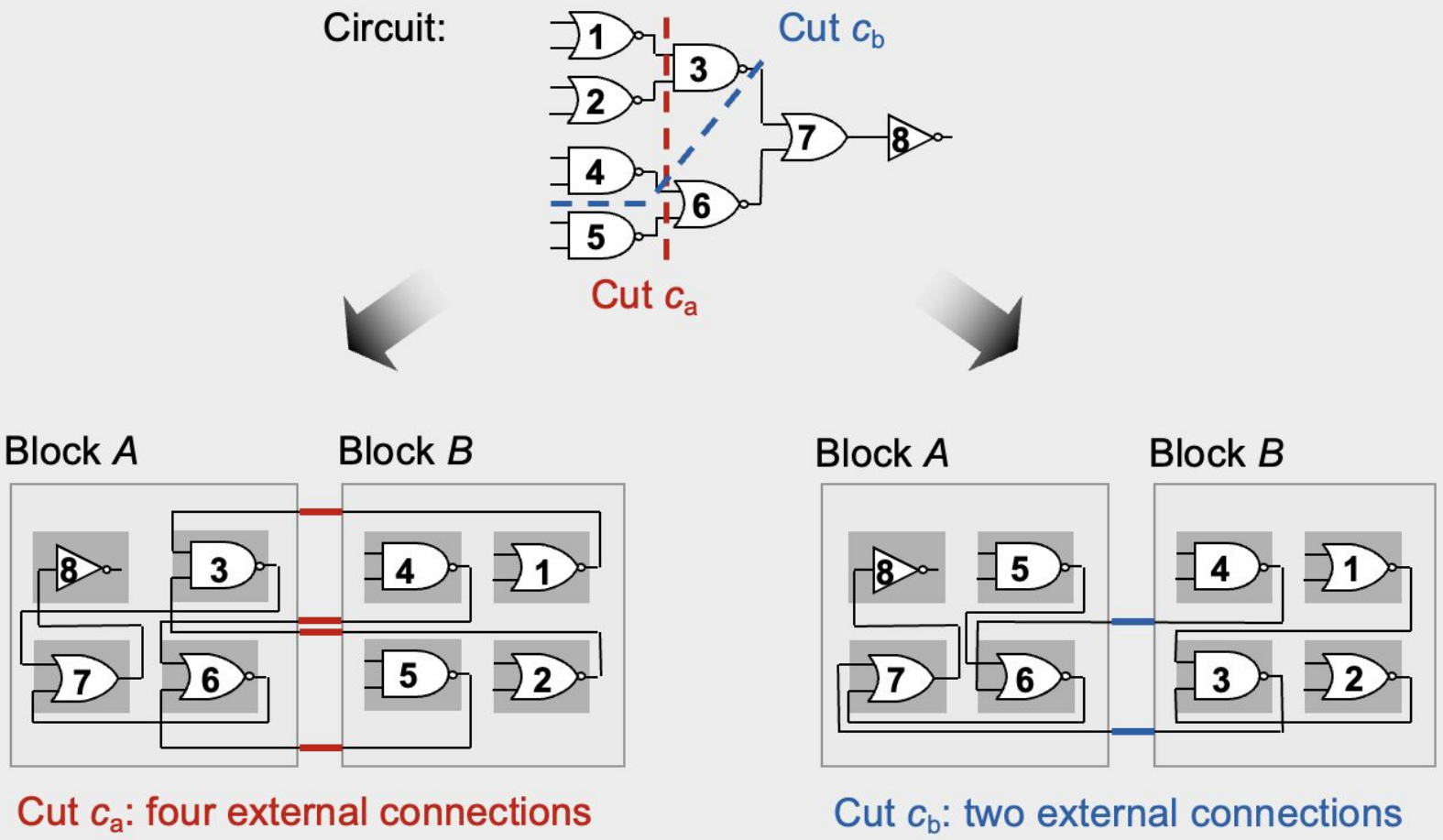
CSE 460: VLSI Design

Lecture 13+14: VLSI Physical Design

VLSI Design Flow

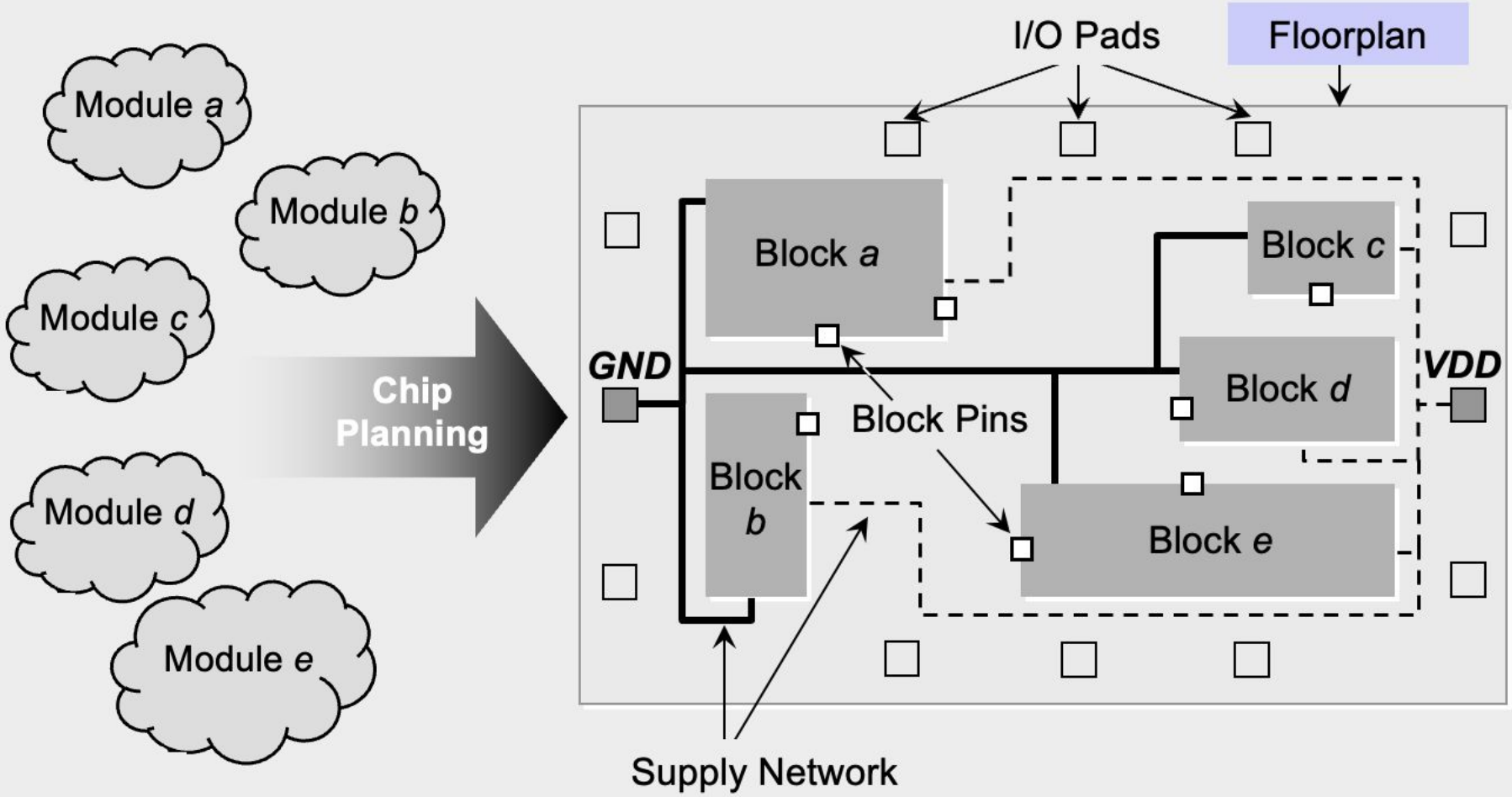


Partitioning



Algorithms Involved: Kernighan-Lin (KL) Algorithm, Fiduccia-Mattheyses (FM) Algorithm

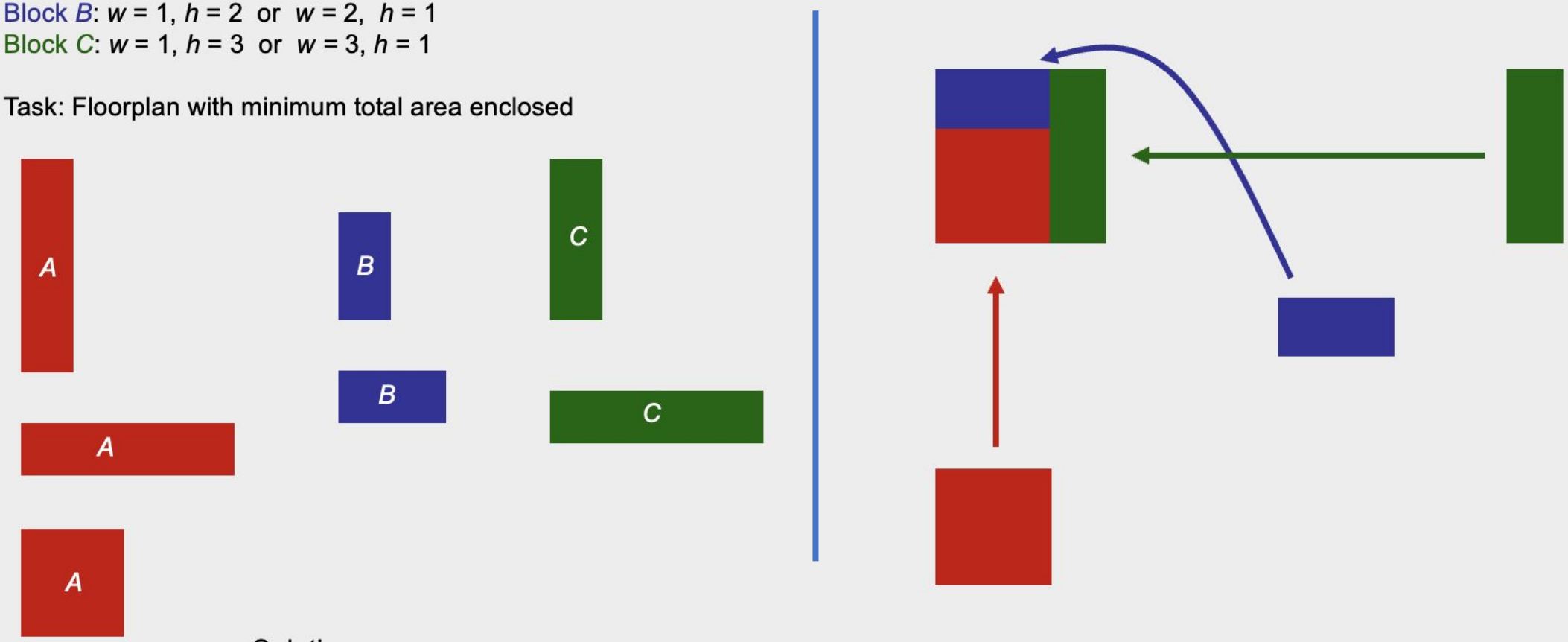
Chip Planning



Chip Planning (Example)

Given: Three blocks with the following potential widths and heights
Block *A*: $w = 1$, $h = 4$ or $w = 4$, $h = 1$ or $w = 2$, $h = 2$
Block *B*: $w = 1$, $h = 2$ or $w = 2$, $h = 1$
Block *C*: $w = 1$, $h = 3$ or $w = 3$, $h = 1$

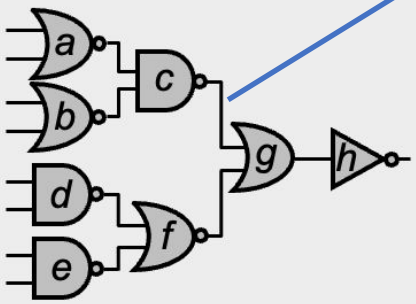
Task: Floorplan with minimum total area enclosed



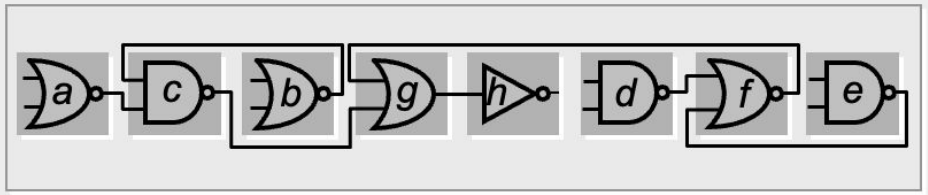
Solution:
Aspect ratios
Block *A* with $w = 2$, $h = 2$; Block *B* with $w = 2$, $h = 1$; Block *C* with $w = 1$, $h = 3$

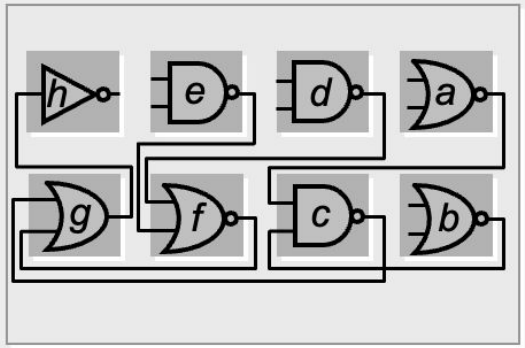
Placement

for simplicity, all gates are inside one single module : assumed

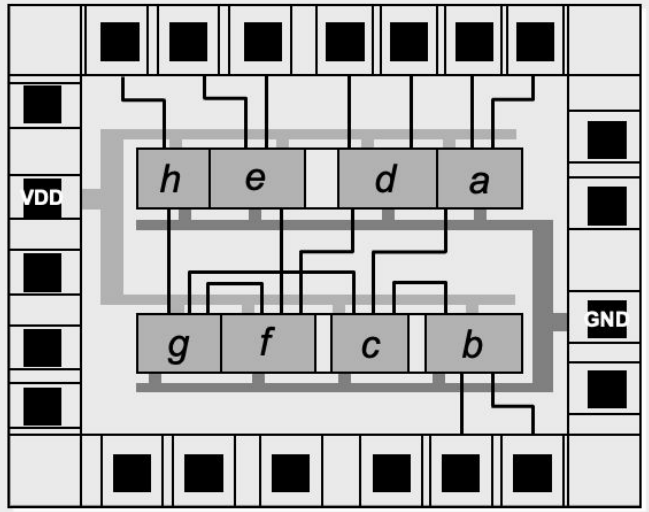


Linear Placement



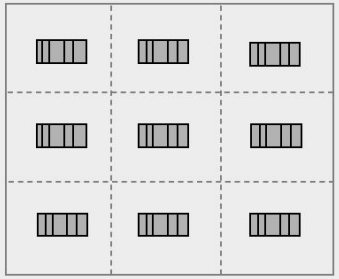


2D Placement

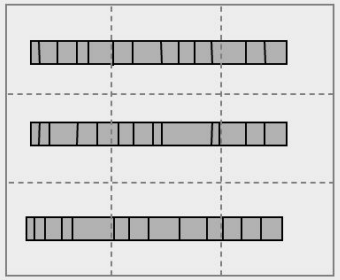


Placement and Routing with Standard Cells

Global Placement



Detailed Placement

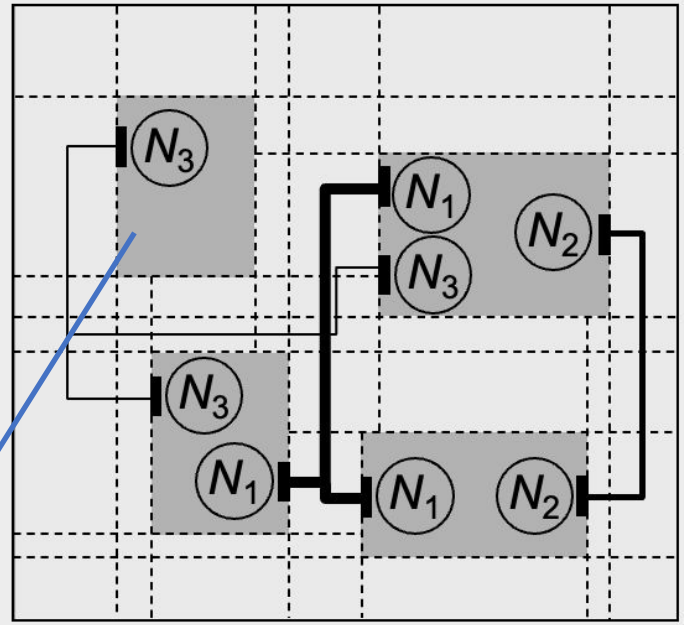


Signal Routing

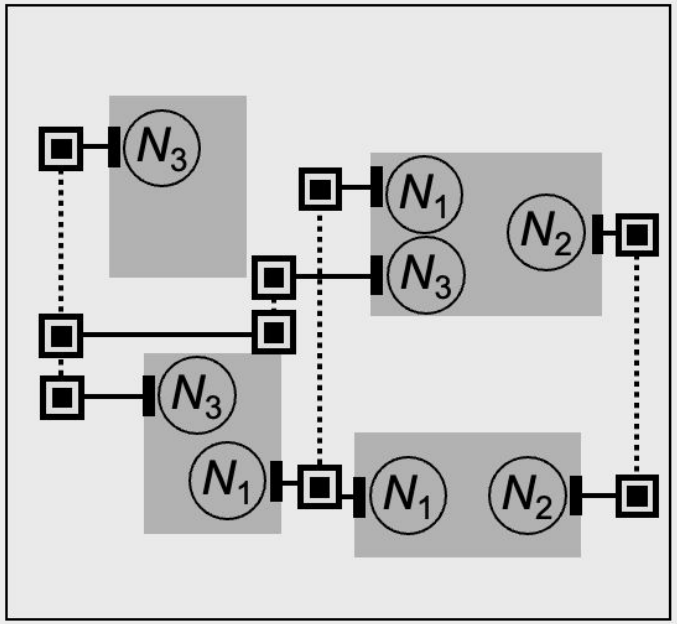
Here, all horizontal connections done by metal -1

All vertical connections done by metal - 2

Global Routing



Detailed Routing



these gray boxes are modules.

N1,N2 etc indicate nodes or logic gates etc inside the module

Horizontal Segment

Vertical Segment

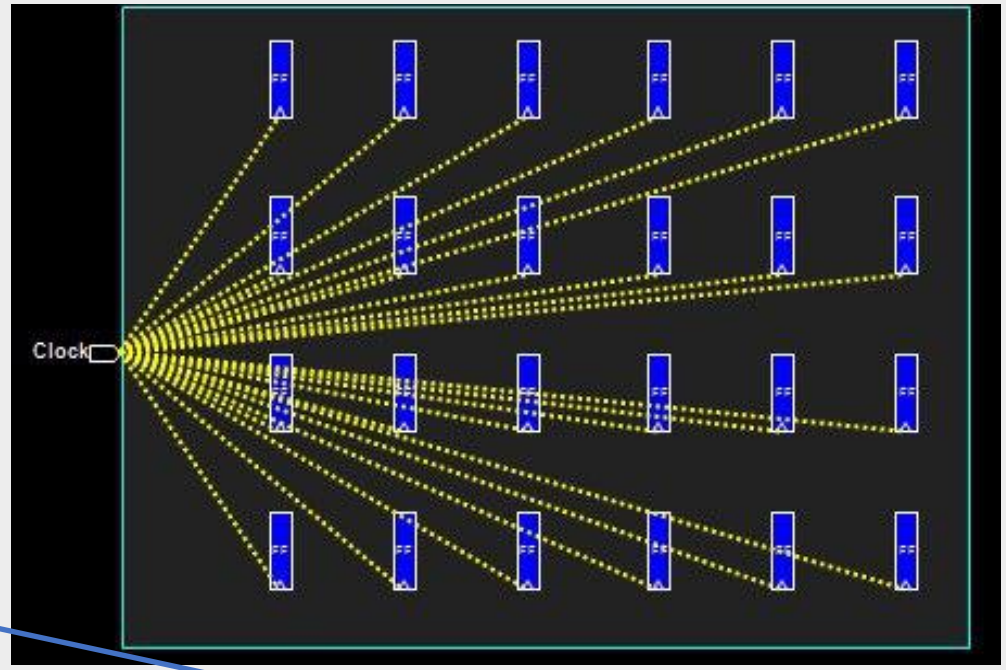
Via

via is used to connect the two metal layers at one point

Clock Tree Synthesis (CTS)

Definition:

- Clock tree synthesis is a process which make sure that the clock gets distributed **evenly** to all sequential elements in a design.
- CTS is the process of **insertion of buffers or inverters** along the clock paths of ASIC design in order to achieve minimum skew or balanced skew.
- In ICs, clock consumes around **half of the total power** consumption. Here clock gating technique helps to reduce power consumption by the clocks.



Clock gating is a technique to reduce power consumption in ICs by selectively disabling the clock signal to parts of the circuit when they are not active. This is achieved using gating logic (e.g., an AND gate or latch or tristate buffer) that controls whether the clock reaches a specific block based on its activity requirement. By stopping the clock signal for idle sections, unnecessary switching is prevented, reducing dynamic power consumption effectively.

Goals of CTS:

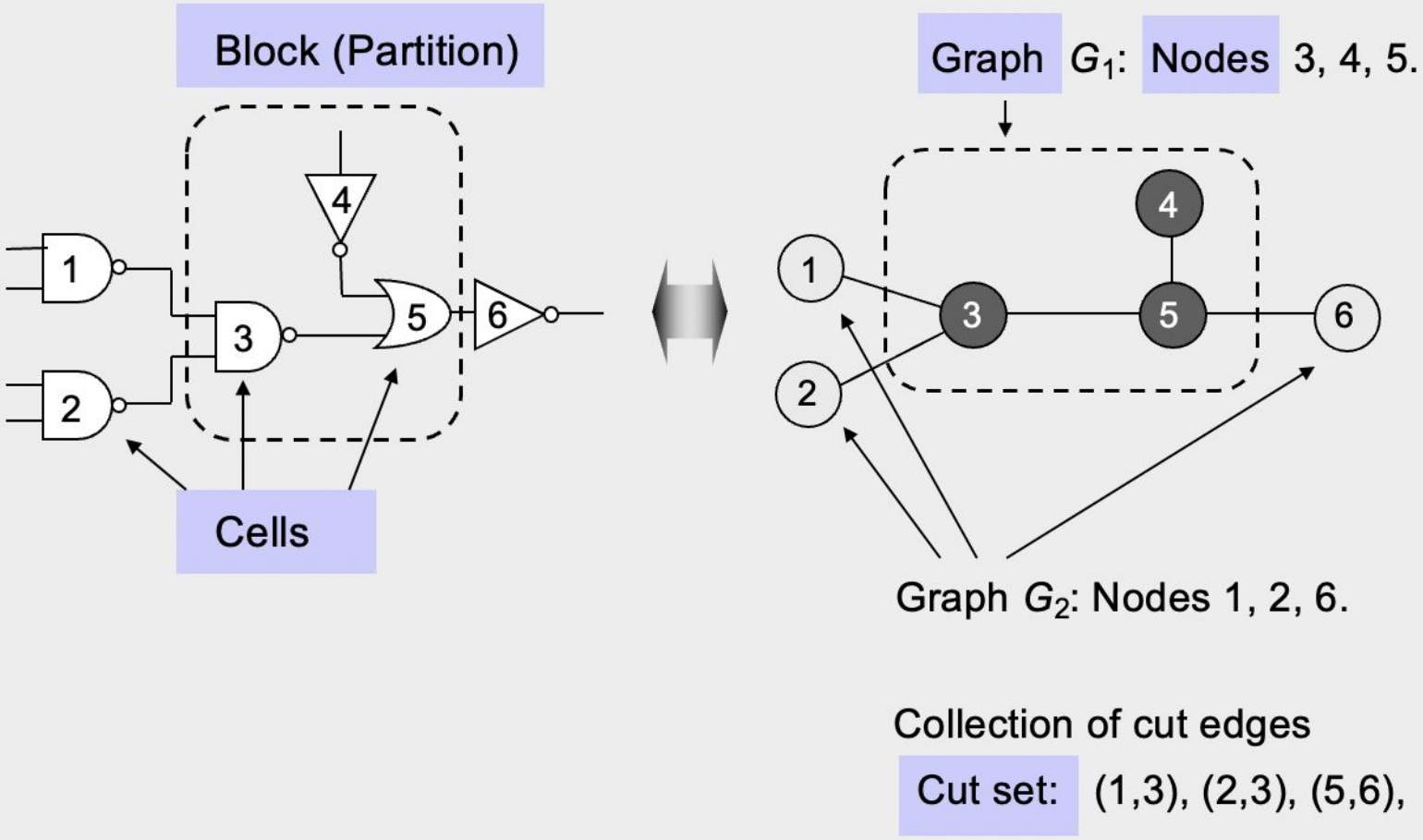
- To meet clock tree design rule constraints such as maximum transition, maximum load capacitance and maximum fanout.
- To meet clock tree targets such as minimum skew and minimum insertion delay.

Timing Closure

- Timing closure is the process by which a logic design consisting of primitive elements such as combinatorial logic gates (and , or , not , nand , nor , etc.) and sequential logic gates (flip flops, latches, memories) is modified to meet its timing requirements.

- Timing closure is done through **layout optimizations and netlist modifications.**

Partition Terminology



Optimization Goals

- Given a graph $G(V,E)$ with $|V|$ nodes and $|E|$ edges where each node $v \in V$ and each edge $e \in E$.
- Each node has area $s(v)$ and each edge has cost or weight $w(e)$.
- The objective is to divide the graph G into k disjoint subgraphs such that all optimization goals are achieved and all original edge relations are respected.

Optimization Goals

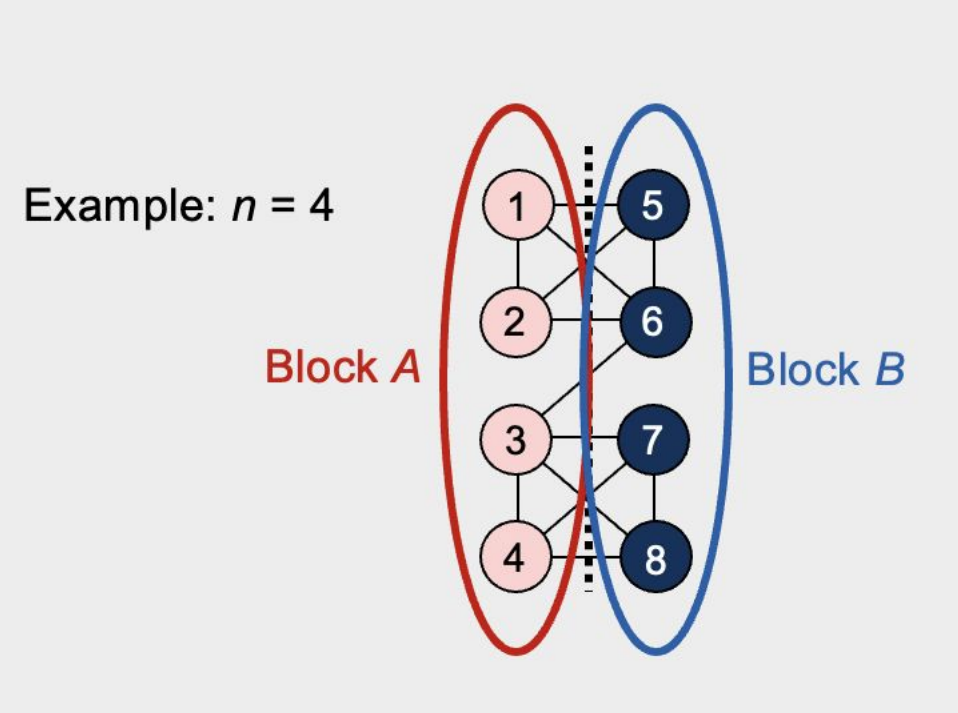
- In detail, what are the optimization goals?
  - Number of connections between partitions is minimized
  - Each partition meets all design constraints (size, number of external connections..)
  - Balance every partition as well as possible

- How can we meet these goals?
  - Unfortunately, this problem is NP-hard
  - Efficient heuristics are developed in the 1970s and 1980s. They are high quality and in low-order polynomial time.

Kernighan Lin (KL Algorithm)

Given: A graph with $2n$ nodes where each node has the same weight.

Goal: A partition (division) of the graph into two disjoint subsets A and B with minimum cut cost and $|A| = |B| = n$.



KL Algorithm (Terminology)

Cost $D(v)$ of moving a node v

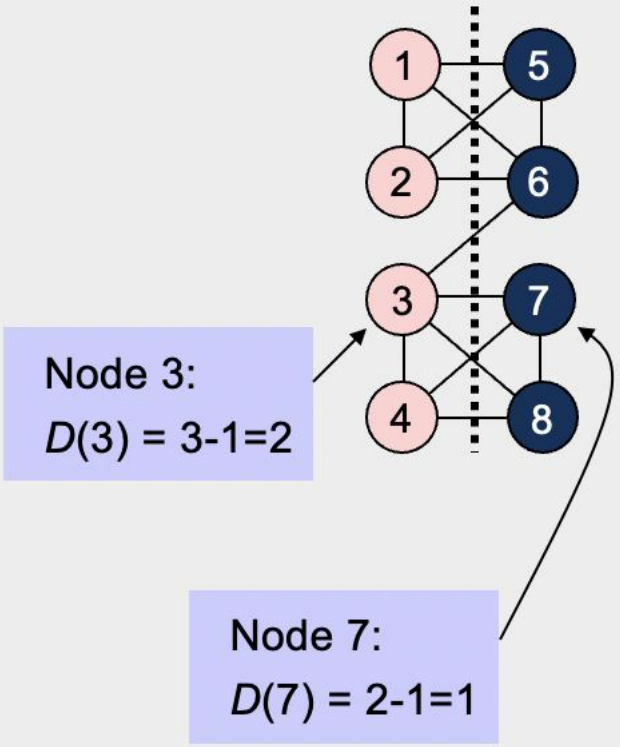
$D(v) = |E_c(v)| - |E_{nc}(v)|$,

where

$E_c(v)$ is the set of v's incident edges that are cut by the cut line, and

$E_{nc}(v)$ is the set of v's incident edges that are not cut by the cut line.

High costs ($D > 0$) indicate that the node should move, while low costs ($D < 0$) indicate that the node should stay within the same partition.



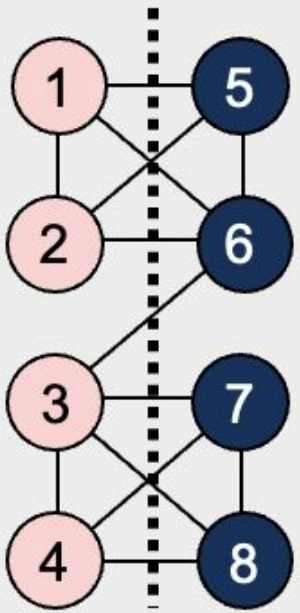
KL Algorithm (Terminology)

Gain of swapping a pair of nodes *a* und *b*

$\Delta g = D(a) + D(b) - 2 \cdot c(a,b)$

where

- $D(a)$, $D(b)$ are the respective costs of nodes a, b
- $c(a,b)$ is the connection weight between a and b:
  If an edge/connection exists between a and b,
  then $c(a,b)$ = edge weight (here 1),
  otherwise, $c(a,b) = 0$.



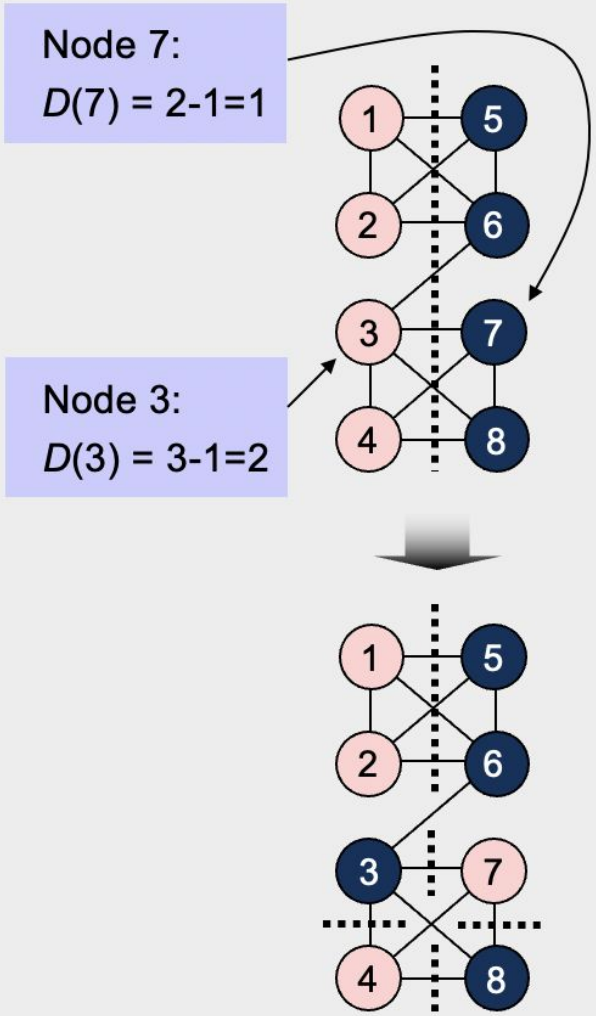
The gain Δg indicates how useful the swap between two nodes will be

The larger Δg, the more the total cut cost will be reduced

KL Algorithm (Terminology)

$\Delta g\ (3,7) = D(3) + D(7) - 2 \cdot c(a,b) = 2 + 1 - 2 = 1$

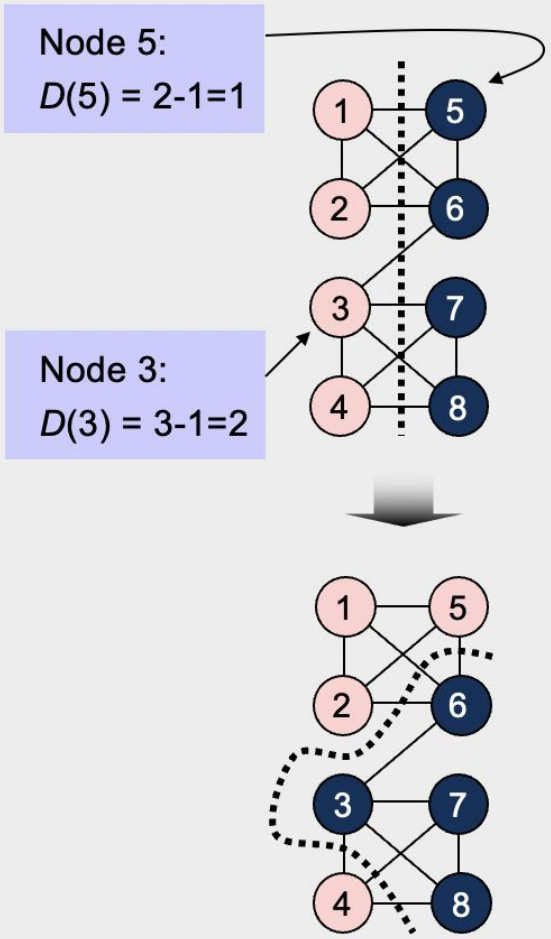
=> Swapping nodes 3 and 7 would reduce the cut size by 1



KL Algorithm (Terminology)

$\Delta g\ (3,5) = D(3) + D(5) - 2 \cdot c(a,b) = 2 + 1 - 0 = 3$

=> Swapping nodes 3 and 5 would reduce the cut size by 3



KL Algorithm (Terminology)

Gain of swapping a pair of nodes *a* und *b*

The goal is to find a pair of nodes *a* and *b* to exchange such that Δg is maximized and swap them.

Maximum positive gain G_m of a pass

The maximum positive gain G_m corresponds to the best prefix of *m* swaps within the swap sequence of a given pass.

These *m* swaps lead to the partition with the minimum cut cost encountered during the pass.

$$G_m = \sum_{i=1}^{m} \Delta g_i$$

G_m is computed as the sum of Δg values over the first *m* swaps of the pass, with *m* chosen such that G_m is maximized.

KL Algorithm (One-pass) - Summary of all steps

Step 0:

- $V = 2n$ nodes
- $\{A, B\}$ is an initial arbitrary partitioning

Step 1:

- $i = 1$
- Compute $D(v)$ for all nodes $v \in V$

Step 2:

- Choose a_i and b_i such that $\Delta g_i = D(a_i) + D(b_i) - 2 * c(a_i b_i)$ is maximized
- Swap and fix a_i and b_i

Step 3:

- If all nodes are fixed, go to Step 4. Otherwise
- Compute and update D values for all nodes that are connected to a_i and b_i and are not fixed.
- $i = i + 1$
- Go to Step 2

Step 4:

- Find the move sequence $1 \ldots m$ ($1 \le m \le i$), such that $G_m = \sum_{i=1}^{m} \Delta g_i$ is maximized
- If $G_m > 0$, go to Step 5. Otherwise, END

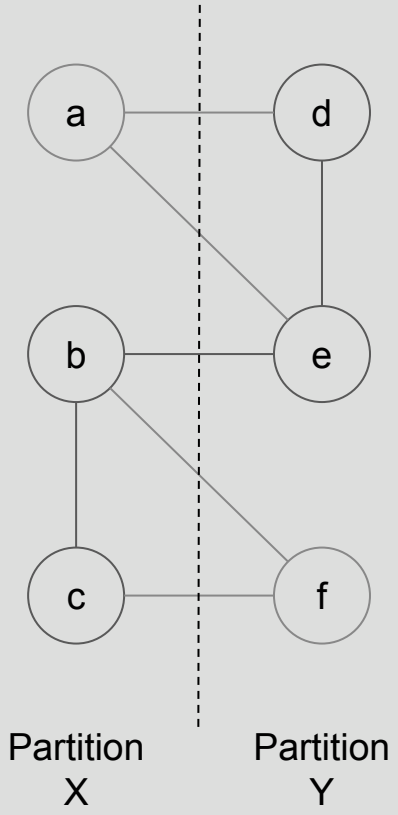
Step 5:

- Execute m swaps, reset remaining nodes
- Go to Step 1

KL Algorithm (Example 1)

The graph (nodes a-f) can be optimally partitioned using the Kernighan-Lin algorithm. The dotted line represents the initial partitioning. Assume all the edges have the same weight.

a. What is the initial cut cost?
b. Perform the first pass of the algorithm.
   [*Hint: For the "i"th iteration of the first pass, until all the nodes are swapped and fixed, do the following:*
   i. *Compute/update the node costs of all unfixed nodes*
   ii. *Find the maximum gain of swapping a pair of nodes (Δg_i)*
   iii. *Swap the pair and draw the updated graph*]
c. Finish the first pass by computing the maximum positive gain, G_m. Suggest how many swaps should be actually executed in the first pass.
d. Should you perform subsequent passes of the algorithm? Why or why not?
e. Perform the higher pass/passes if applicable.

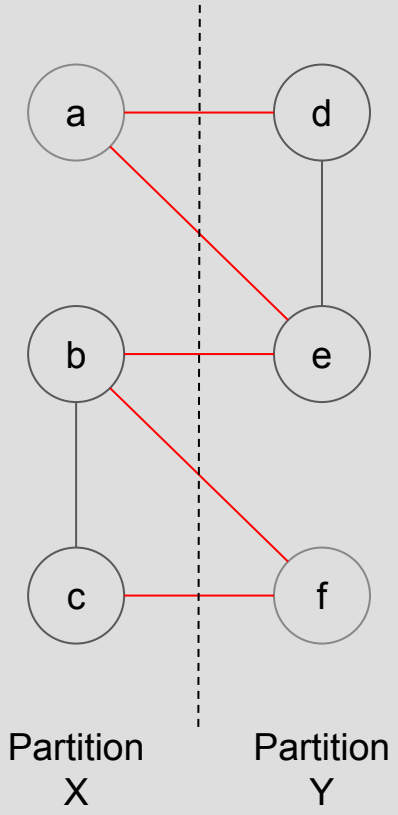


KL Algorithm (Example 1)

The graph (nodes a-f) can be optimally partitioned using the Kernighan-Lin algorithm. The dotted line represents the initial partitioning. Assume all the edges have the same weight.

a. What is the initial cut cost?

Ans: We can see that the initial partition line intersects 5 edges of the graph (shown in **red**). So the initial cut cost is **5**.



KL Algorithm (Example 1)

The graph (nodes a-f) can be optimally partitioned using the Kernighan-Lin algorithm. The dotted line represents the initial partitioning. Assume all the edges have the same weight.

b. Perform the first pass of the algorithm.

For the "i=1" iteration of the first pass:

i. Compute the node costs of all unfixed nodes

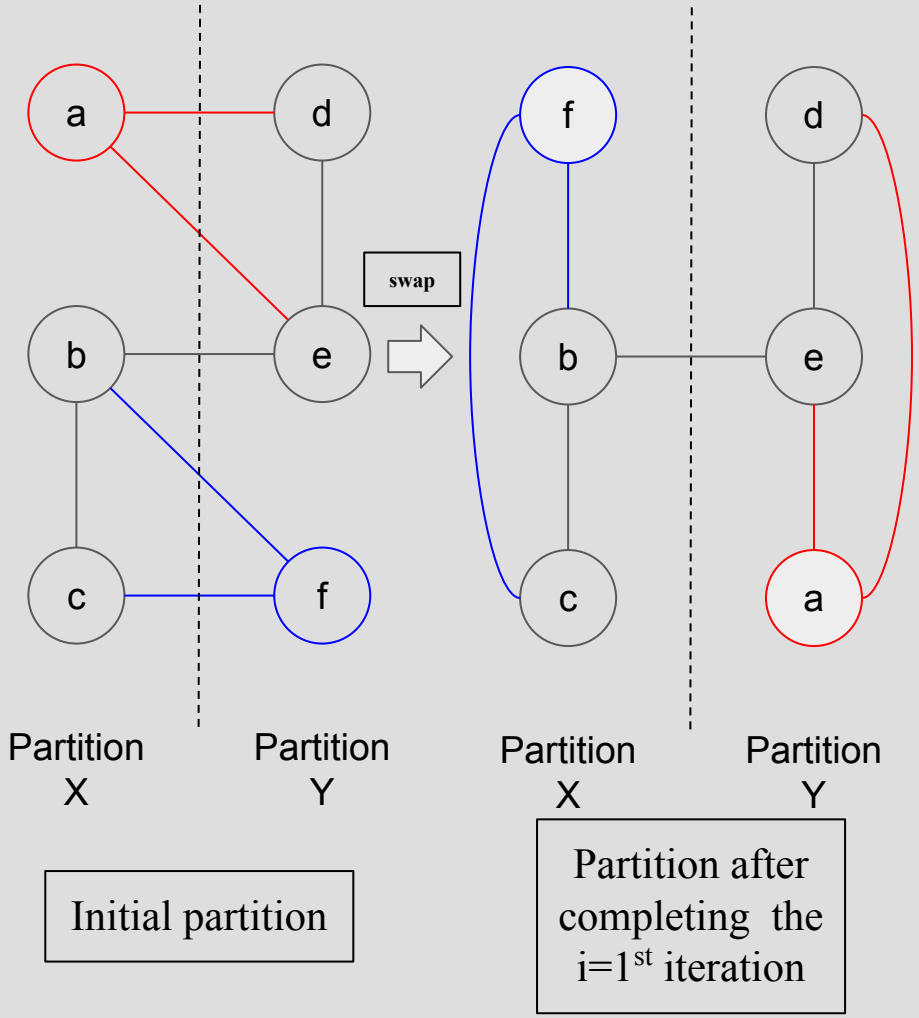
Ans: D(a) = 2 - 0 = 2; D(b) = 2 - 1 = 1; D(c) = 1 - 1 = 0;
D(d) = 1 - 1 = 0; D(e) = 2 - 1 = 1; D(f) = 2 - 0 = 2.

ii. Find the maximum gain of swapping a pair of nodes (Δg_i)

Ans: $\Delta g_1(a, f) = 2 + 2 - 0 = 4$.

iii. Swap the pair and draw the updated graph

Ans: Swap nodes a & f, and <u>fix them</u>.



KL Algorithm (Example 1)

b. Perform the first pass of the algorithm.

For the "i=2" iteration of the first pass:

i. Update the node costs of all unfixed nodes

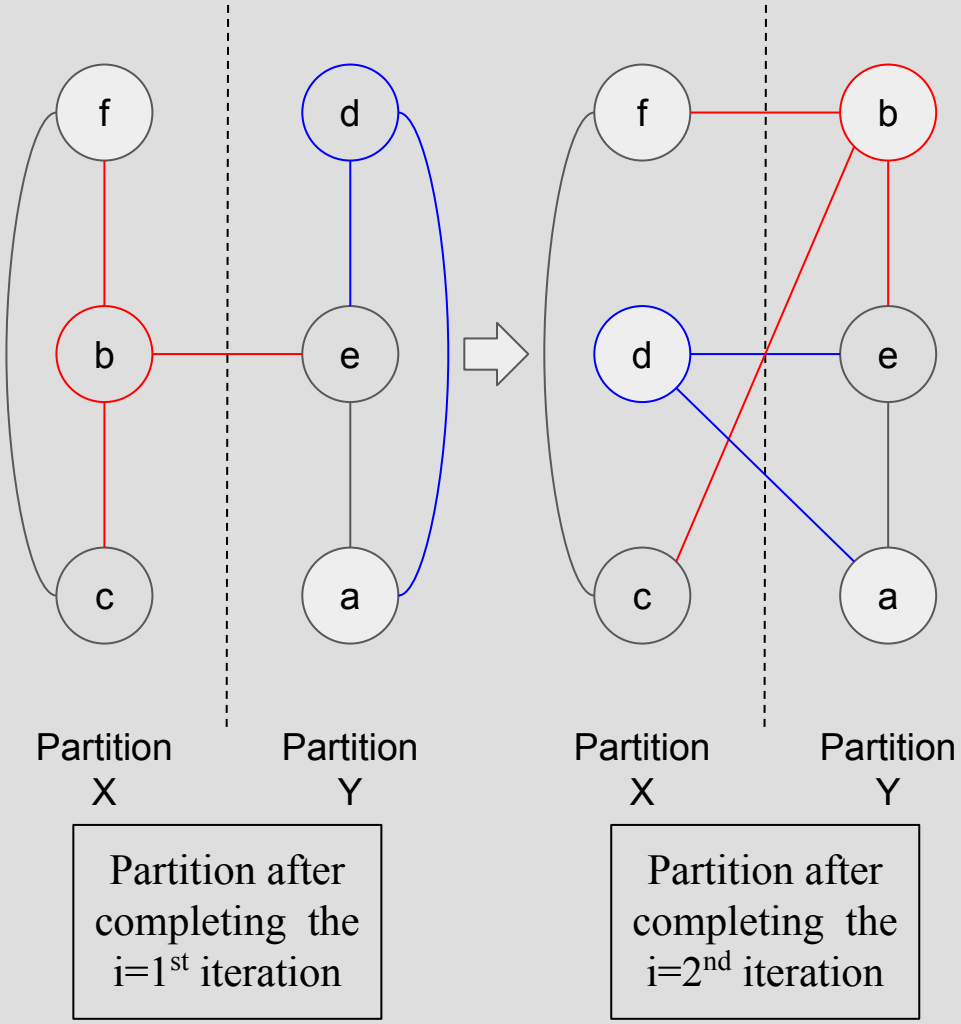
Ans: ~~D(a) = 2 - 0 = 2~~; D(b) = 1 - 2 =-1; D(c) = 0 - 2 =-2;
D(d) = 0 - 2 =-2; D(e) = 1 - 2 =-1; ~~D(f) = 2 - 0 = 2~~.

ii. Find the maximum gain of swapping a pair of nodes (Δg_i)

Ans: Δg_2(b, d) = -1 -2 -0 = -3 or Δg_2(c, e) = -2 -1 -0 = -3

iii. Swap the pair and draw the updated graph

Ans: Swap nodes b & d, and <u>fix them</u> or c & e, and fix them.
[Both approaches are valid and will end in the same result.]



KL Algorithm (Example 1)

b. Perform the first pass of the algorithm.

For the "i=3" iteration of the first pass:

i. Update the node costs of all unfixed nodes

Ans: ~~D(a) = 2 - 0 = 2~~; ~~D(b) = 1 - 2 = -1~~; D(c) = 1 - 1 = 0;

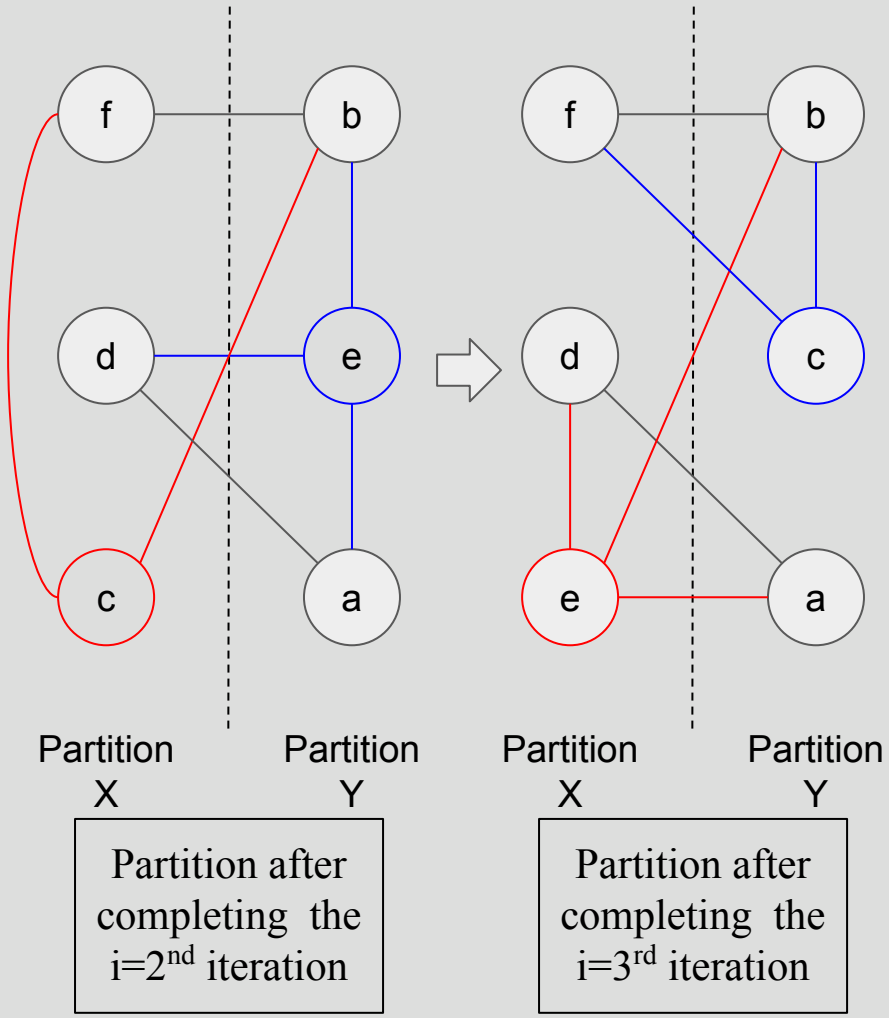
~~D(d) = 1 - 1 = 0~~; D(e) = 1 - 2 =-1; ~~D(f) = 2 - 0 = 2~~.

ii. Find the maximum gain of swapping a pair of nodes (Δg_i)

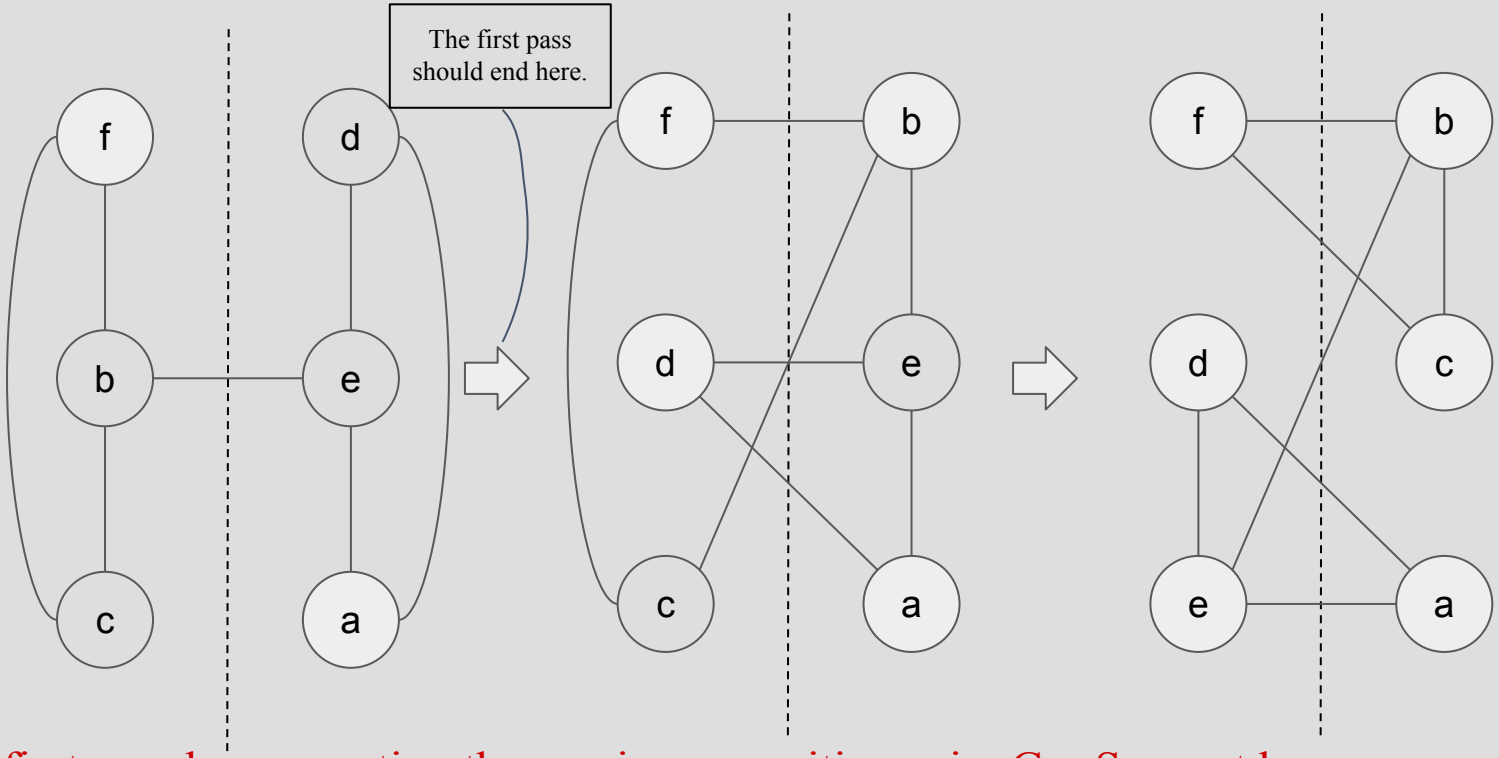
Ans: $\Delta g_3(c, e) = 0 - 1 - 0 = -1$

iii. Swap the pair and draw the updated graph

Ans: Swap nodes c & e, and <u>fix them</u>.



KL Algorithm (Example 1)



c. Finish the first pass by computing the maximum positive gain, G_m. Suggest how many swaps should be actually executed in the first pass.

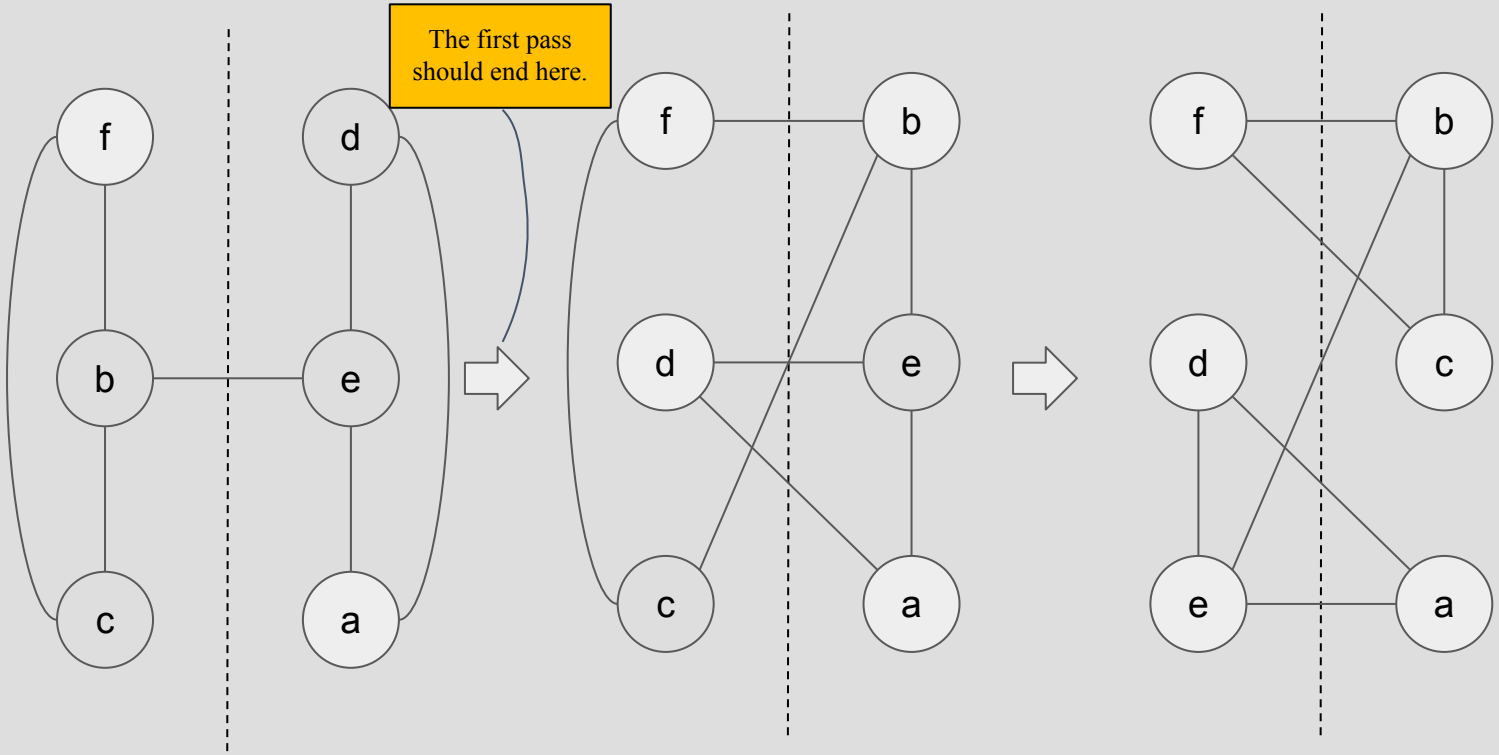
$G_1 = \Delta g_1 \quad = 4 \quad = 4$

$G_2 = \Delta g_1 + \Delta g_2 \quad = 4 - 3 \quad = 1$

$G_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = 4 - 3 - 1 = 0$

Hence, maximum positive gain is achieved for m = 1. Only the swap for Δg_1, or (a, f) should be executed.

KL Algorithm (Example 1)



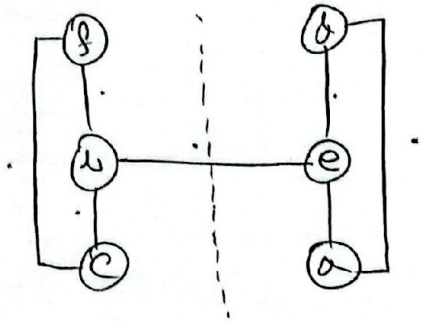
d. Should you perform subsequent passes of the algorithm? Why or why not?

Ans: Since $G_m > 0$, subsequent passes should be performed (until $G_m \leq 0$).

e.

Second Pass

First Iteration



$D(f) = 0 - 2 = -2$; $D(b) = 1 - 2 = -1$; $D(c) = 0 - 2 = -2$

$D(b) = 0 - 2 = -2$; $D(e) = 1 - 2 = -1$; $D(a) = 0 - 2 = -2$

$g(f,b) = -2 - 2 - 0 = -4$; $g(f,e) = -2 - 1 - 0 = -3$

$g(f,a) = -2 - 2 - 0 = -4$; $g(b,b) = -1 - 2 - 0 = -3$

$g(b,e) = -1 - 1 - 2 = -4$; $g(b,a) = -1 - 2 - 0 = -3$

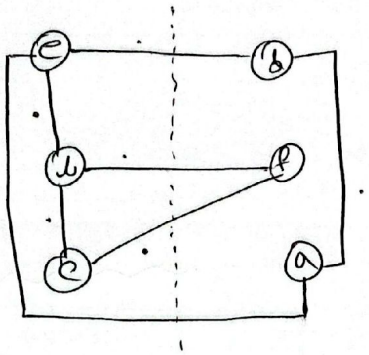
$g(c,b) = -2 - 2 - 0 = -4$; $g(c,e) = -2 - 1 - 0 = -3$

$g(c,a) = -2 - 2 - 0 = -4$

$\Delta g1 = -3$

Swap nodes f and e

Second Iteration



$D(d) = 1 - 2 = -1$

$D(b) = 1 - 1 = 0$

$D(c) = 1 - 1 = 0$

$D(a) = 1 - 1 = 0$

$g(d,b) = -1 + 0 - 0 = -1$

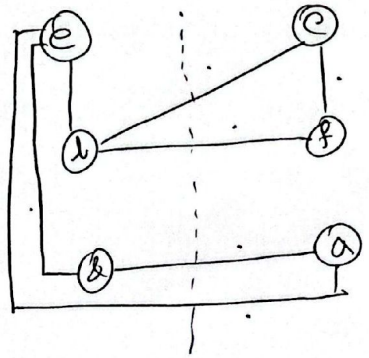
$g(c,b) = 0 + 0 - 0 = 0$

$g(d,a) = -1 + 0 - 0 = -1$

$g(c,a) = 0 + 0 + 0 = 0$

$g_2 = 0$

Swap c and b

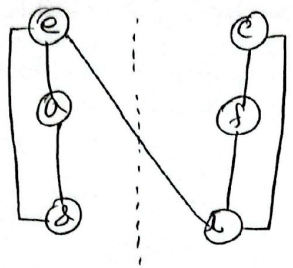


Third Iteration

$D(b) = 2 - 1 = 1$ $\qquad$ $D(a) = 2 - 0 = 2$

$\Delta g_3(b, a) = 1 + 2 - 0 = 3$

swap nodes b and a.



$G_1 = \Delta g_1 = -3$

$G_2 = \Delta g_1 + \Delta g_2 = -3 + 0 = -3$

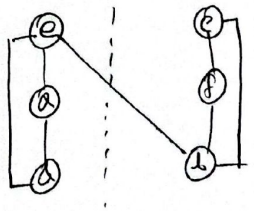
$G_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = -3 + 3 = 0$

$G_m = 0$

So all the nodes should be swapped, 3 swaps should be done.

As $G_m \leq 0$; no need of additional pass.

Final partition



Initial cut cost = 5

After partitioning cut cost = 1

So cut cost decreased, partition is justified.

Significance of Cumulative gain, G:

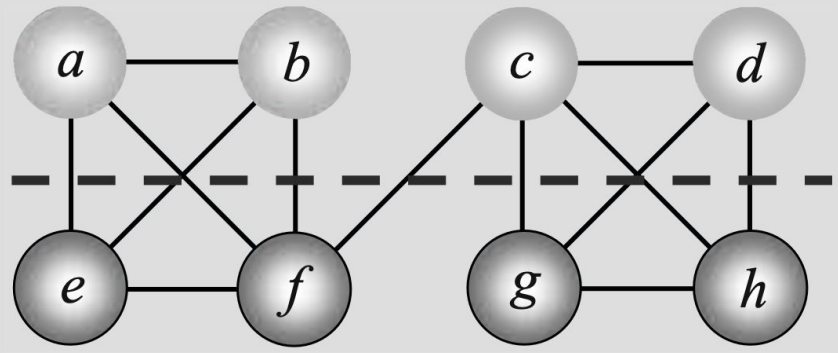
1.Tells us how many swaps should actually be executed; When we achieve max cumulative gain Gm, we need to stop at that iteration. Suppose, you first performed 5 iterations in 1st pass and you get max cumulative gain, Gm == G3; hence, you need to perform actually 3 swaps for optimum result in first pass.

2.Determining the max cumulative gain, Gm, we decide whether we need to perform 2nd pass or not.

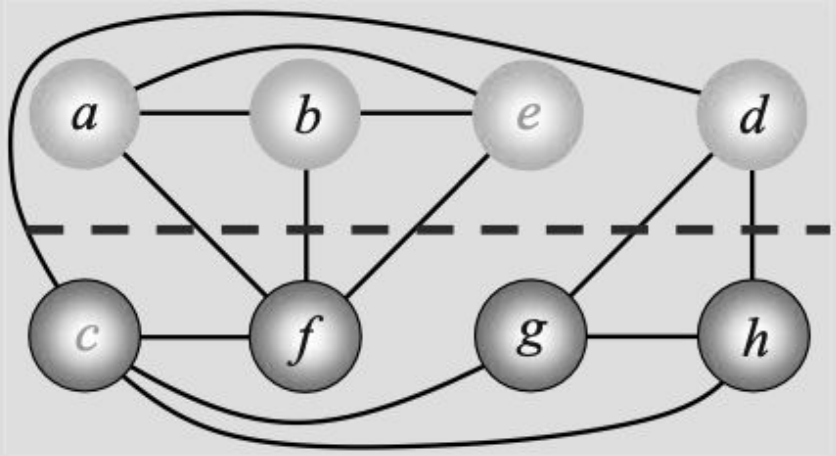
3.Tells us how many units of cut-cost have been reduced after a particular i-th iteration (final cut-cost=initial – Gi).

KL Algorithm (Example 2)

Given: initial partition of nodes *a-h* (right).
Task: perform the first pass of the KL algorithm.



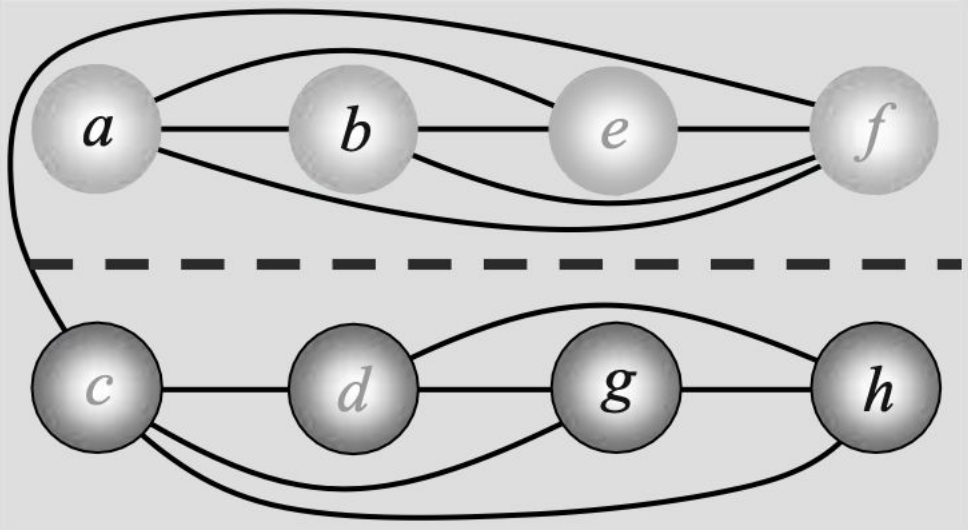
Solution:
Initial cut cost = 9.
Compute $D(v)$ costs for all free nodes *a-h*.
$D(a) = 1$, $D(b) = 1$, $D(c) = 2$, $D(d) = 1$,
$D(e) = 1$, $D(f) = 2$, $D(g) = 1$, $D(h) = 1$
$\Delta g_1 = D(c) + D(e) - 2c(c,e) = 2 + 1 - 0 = 3$
Swap and fix nodes *c* and *e*.



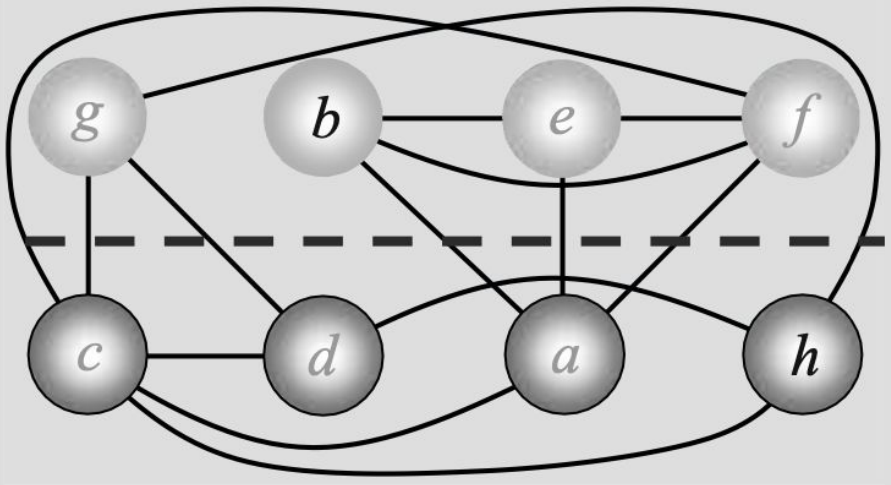
Update $D(v)$ costs for all free nodes connected to newly swapped nodes *c* and *e*: *a*, *b*, *d*, *f*, *g* and *h*.
$D(a) = -1$, $D(b) = -1$, $D(d) = 3$,
$D(f) = 2$, $D(g) = -1$, $D(h) = -1$
$\Delta g_2 = D(d) + D(f) - 2c(d,f) = 3 + 2 - 0 = 5$
Swap and fix nodes *d* and *f*.

KL Algorithm (Example 2)

Given: initial partition of nodes *a*-*h* (right).
Task: perform the first pass of the KL algorithm.



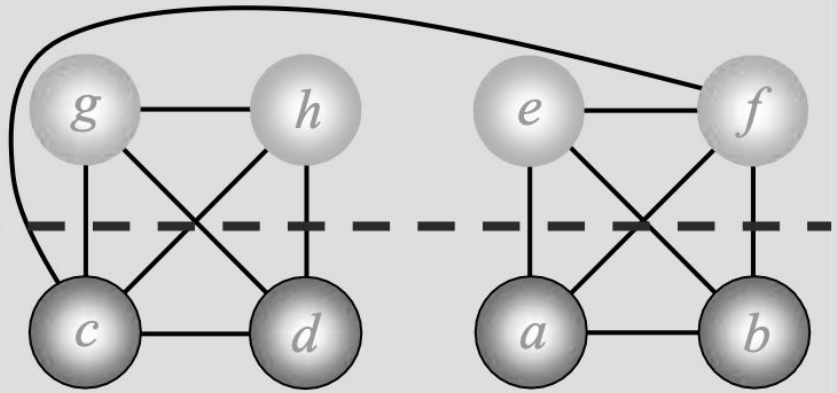
Update $D(v)$ costs for all free nodes connected to newly swapped nodes *d* and *f*: *a*, *b*, *g* and *h*.
$D(a) = -3$, $D(b) = -3$, $D(g) = -3$, $D(h) = -3$
$\Delta g_3 = D(a) + D(g) - 2c(a,g) = -3 + -3 - 0 = -6$
Swap and fix nodes *a* and *g*.

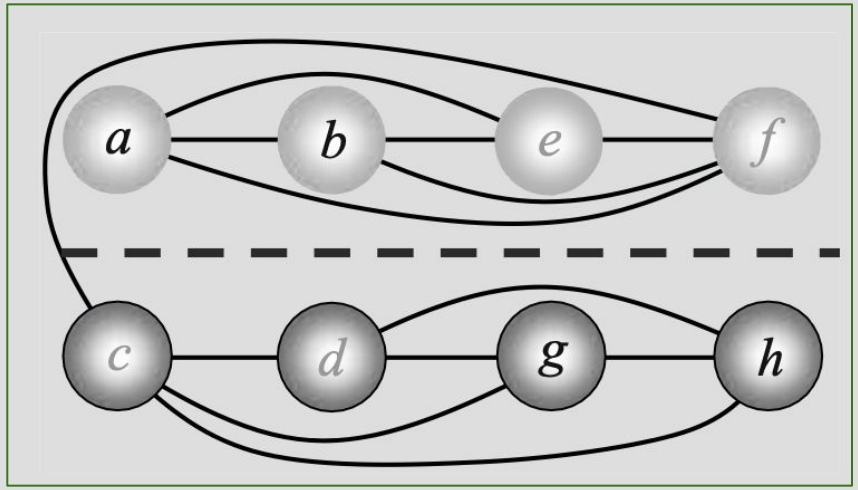


Update $D(v)$ costs for all free nodes connected to newly swapped nodes *a* and *g*: *b* and *h*.
$D(b) = -1$, $D(h) = -1$
$\Delta g_4 = D(b) + D(h) - 2c(b,h) = -1 + -1 - 0 = -2$
Swap and fix nodes *b* and *h*.

KL Algorithm (Example 2)

Given: initial partition of nodes *a*-*h* (right).
Task: perform the first pass of the KL algorithm.





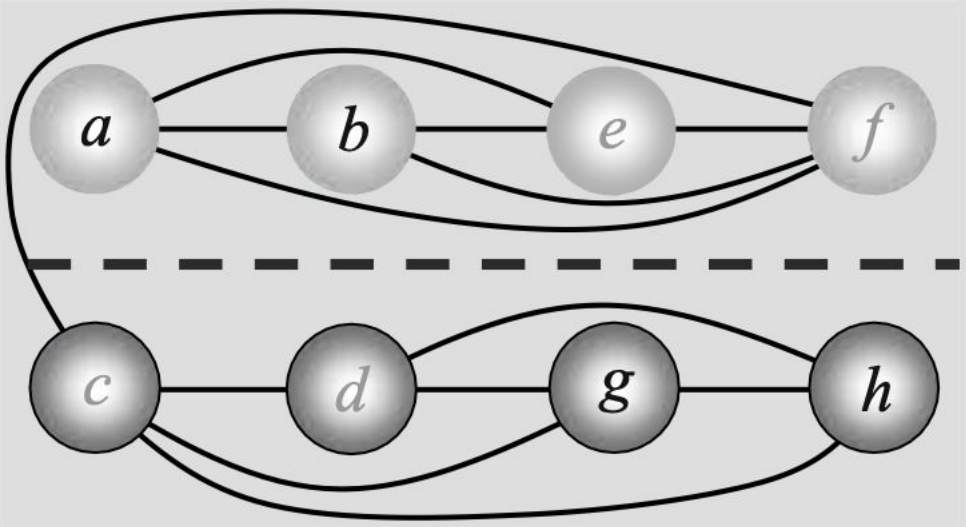
Compute maximum positive gain G_m.

$$G_1 = \Delta g_1 = 3$$
$$G_2 = \Delta g_1 + \Delta g_2 = 3 + 5 = 8$$
$$G_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = 3 + 5 + -6 = 2$$
$$G_4 = \Delta g_1 + \Delta g_2 + \Delta g_3 + \Delta g_4 = 3 + 5 + -6 + -2 = 0$$

$G_m = 8$ with $m = 2$. Since $G_m > 0$, the first $m = 2$ swaps are executed: (c,e) and (d,f). Additional passes are performed until $G_m \leq 0$.

Final Cut-cost after two swaps= initial cutcost- G2 = 9-8=1

KL Algorithm (Example 2)

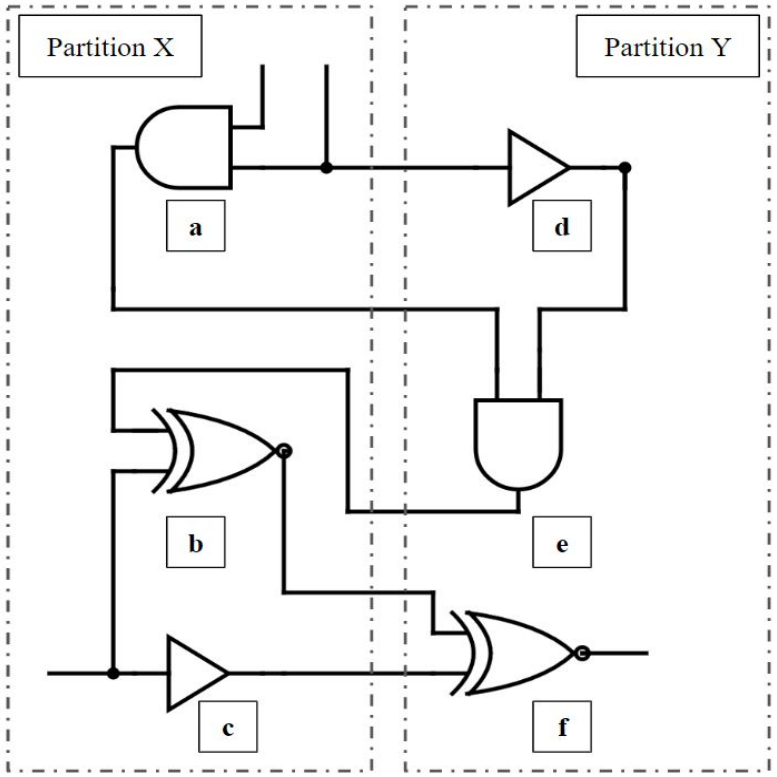


Now, execute the 2nd pass taking this arrangement and finally verify whether you are getting maximum cumulative gain, **Gm<= 0** for this pass. If not, you need to go to 3rd pass.

Otherwise, the partition that you have taken at the beginning of this pass is the optimum partition result.

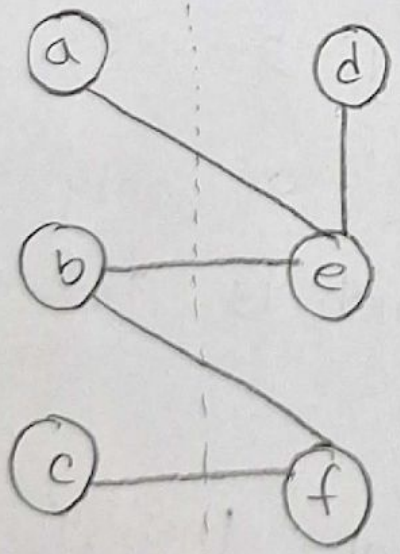
Example-3

Consider the following circuit with 6 logic gates (a-f) and the initial partitioning marked by the dotted lines.



(a)	**Represent** the given circuit and the initial partitioning using a graph and find initial cut-cost.
(b)	**Perform** the first pass of the algorithm. For every "i"th iteration inside the first pass, until all the nodes are swapped and fixed, do the following: i. Find the maximum gain of swapping a pair of nodes (Δg_i) ii. Swap the pair and draw the updated graph
(c)	Finish the first pass by computing the maximum positive gain, G_m. **Suggest** how many swaps should be **actually executed** in the first pass.
(d)	Should you perform subsequent passes of the algorithm? **Why or why not**?

(a)



So the initial cut cost = 4.

(b)

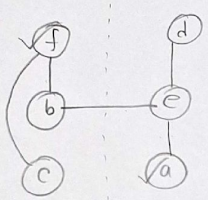
$D(a) = 1$, $D(b) = 2$, $D(c) = 1$, $D(d) = -1$ $D(e) = 1$ $D(f) = 2$

$\Delta g(a,e) = 0$ $\Delta g(b,d) = 1$. $\Delta g(c,d) = 0$

$\Delta g(a,d) = 0$ $\Delta g(b,e) = 1$ $\Delta g(c,e) = 2$

$\Delta g(a,f) = 3$ $\Delta g(b,f) = 2$ $\Delta g(c,f) = 1$.

~~$\Delta g_1 = (a$~~ $\Delta g_1(a,f) = 3$ (max). So swapping a, f.



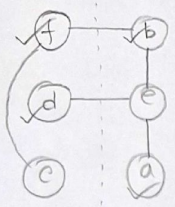
Here,

$D(b) = 0$, $D(c) = -1$, $D(d) = -1$, $D(e) = \cancel{0}\ -1$

$\Delta g(b,d) = -1$ $\quad \Delta g(c,d) = -2$

$\Delta g(b,e) = -3$ $\quad \Delta g(c,e) = -2$

$\Delta g_2(b,d) = -1$ (max). swapping b,d.

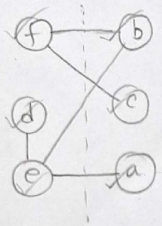


$D(c) = -1$ $\quad D(e) = -1$

$\cancel{\Delta g(c,e) = -2}$ $\quad \Delta g(c,e) = -2$

swapping (c,e).

$\Delta g_3(c,e) = -2$



(c) $Gm_1 = \Delta g_1 = 3$

$Gm_2 = \Delta g_1 + \Delta g_2 = 3 - 1 = 2$

$Gm_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = 3 - 1 - 2 = 0$

Max $Gm_1 = 3$, where $m = 1$. So only the first swap should be executed which is a, f.

And, the final cutcost after 1 swap= 4-G1=3

(d) We know, if $Gm_i \leq 0$ it stops the algorithm and if $Gm_i > 0$ it goes to the next pass. Here $Gm_1 = 3 > 0$. So we should perform subsequent passes of the algorithm. We should go to the next pass.